

DartCodeAI Level 4 ML Systems Practical

Complete Technical Briefing & Study Guide

For the 45-minute live engineering and business review.

1. Part A: Dart Gateway Normalizer

A1: normalizeSuccess - How it works

Two provider response formats must produce one standard AiRuntimeSuccess envelope.

The two formats:

Field	OpenAI format	Internal format
Text	choices[0].message.content	output.text
Prompt tokens	usage.prompt_tokens (snake)	usage.promptTokens (camel)
Completion tokens	usage.completion_tokens	usage.completionTokens
Total tokens	usage.total_tokens	usage.totalTokens
Finish reason	choices[0].finish_reason	finishReason (top-level)

Design decision - structural detection:

Check for "choices" key (OpenAI) vs "output" key (internal). No provider-type parameter exists in the method signature, so payload shape is the distinguishing signal. If neither key exists, throw FormatException.

Why not try one format and fall back to the other?

That would silently mask errors. If an OpenAI response is missing choices, that's malformed - we want it to fail loud, not accidentally get parsed as internal format.

Key helper - _requireInt:

Strict validation that token values are actual integers. If prompt_tokens is "unknown" (malformed fixture), throws FormatException immediately rather than silently coercing to zero.

A2: normalizeFailure - The error taxonomy

Order of checks matters:

- Payload error codes first (content_filtered, capability_unsupported) - providers may signal these alongside various HTTP status codes
- Timeout check (timedOut boolean or HTTP 504) - timeout might have no HTTP status code at all
- HTTP status codes (429, 413, 401/403, 5xx range)
- Fallthrough to invalidProviderResponse - anything we can't map

Upstream	Canonical code	HTTP	Retry	Failover
429	rate_limited	429	yes	no
413	context_length_exceeded	413	no	no
401/403	auth_error	401	no	no
content filter	content_filtered	422	no	no
capability unsp.	capability_unsupported	422	no	no
timeout/504	timeout	504	yes	yes
5xx	provider_unavailable	503	yes	yes

other/null	invalid_provider_response	502	no	no
------------	---------------------------	-----	----	----

[!] CRITICAL: Every error message is a hardcoded static string. NEVER include payload content. The content_filter fixture has "authorization": "Bearer do-not-copy-this-field" - this must NEVER appear in any output.

A3: decideFallback - The three gates

Fallback permitted ONLY when ALL three conditions are true:

- A different fallback provider/model is configured
- Fallback has not already been attempted (at most once)
- Error is timeout or providerUnavailable (only these two have failoverEligible: true)

Defense-in-depth:

Check BOTH failure.failoverEligible AND explicitly verify the code is one of the two permitted codes. If these ever get out of sync, the stricter check wins.

Why no fallback for rate limits?

Rate limiting is a quota issue, not a provider outage. Falling back to provider B won't help if you're also rate-limited there. You'd burn fallback budget on a non-outage.

A4: cosineSimilarity

Formula: $\cos(\theta) = \frac{\text{dot}(A,B)}{(\|A\| \times \|B\|)}$

Validation order: empty vectors -> dimension mismatch -> non-finite values (NaN, Inf) during loop -> zero-norm after loop. Never pad or truncate.

Key values: identical=1.0, opposite=-1.0, orthogonal=0.0

2. Part B: Python Trace Analyzer

Nearest-rank percentile (why this matters)

The spec EXPLICITLY says nearest-rank, not linear-interpolation.

Nearest-rank formula: rank = $\text{ceil}(P/100 \times N)$, take value at that rank (1-indexed)

Always returns an actual observed value. Linear interpolation returns synthetic values between observations. Python's `statistics.quantiles()` uses linear interpolation by default - using it gives WRONG answers.

The actual computed numbers

Metric	Value	How to verify
request_count	20	Count CSV rows
success_rate	0.8	16 of 20 are status 200
error_rate	0.2	4 errors (2x503 + 2x429)
p50 latency	1210ms	10th value sorted = r018
p95 latency	2810ms	19th value = r010
p99 latency	2960ms	$\text{ceil}(0.99 \times 20) = 20\text{th} = \text{r012}$
max latency	2960ms	r012
prompt tokens	1987	Sum prompt_tokens column
completion tokens	762	Sum completion_tokens
total tokens	2749	1987 + 762
fallback_count	7	Count fallback_used=true
fallback_rate	0.35	7 / 20
observed_rps	9.3023	$20 / (2150/1000) = 20/2.15$
tenant-a	12 req, 4 err	All 4 errors are tenant-a
tenant-b	4 req, 0 err	r003, r008, r013, r017
tenant-c	4 req, 0 err	r005, r010, r015, r019

Key design decisions

- Error = non-2xx status codes (`status_code < 200` or `>= 300`)
- `observed_rps = request_count / ((max_elapsed - min_elapsed) / 1000)`
- `_require_int` throws `ValueError` with field name and 1-indexed row number
- Determinism: dicts sorted by key + `json.dumps(sort_keys=True)`

3. Part C: Incident Analysis - Facts vs Inference

FACTS (directly observable in CSV):

- queue_depth ramps 2 -> 45 -> 4 over the trace window
- Two 503s at r006 (depth=19) and r009 (depth=31), both tenant-a
- Two 429s at r011 (depth=38) and r014 (depth=45), both tenant-a
- All 4 errors are tenant-a (which has 60% of requests)
- 7 fallback requests, all hosted-fallback, latencies 2290-2960ms
- Fallback latencies all exceed 2000ms p95 SLO target
- After peak, latencies recover: 1710 -> 1210 -> 820 -> 610ms
- Error requests have 0 completion tokens

INFERENCE (hypothesis):

- Primary provider became overloaded (we see symptoms, not root cause)
- 429s are downstream effect of same overload (correlation, not proven)
- tenant-a's 60% share amplified overload (could be correlation)
- The incident was self-resolving (queue subsided, unknown why)

UNKNOWNNS:

- Whether queue_depth is gateway's queue or provider metric
- What caused the initial queue buildup
- Whether 429s are gateway-enforced or provider-returned
- Whether retries created a feedback loop
- hosted-fallback's normal baseline latency

The 4 errors in the trace:

Request	Status	Tenant	Queue Depth	Latency
r006	503	tenant-a	19	1850ms
r009	503	tenant-a	31	1990ms
r011	429	tenant-a	38	740ms
r014	429	tenant-a	45	690ms

The 7 fallback requests:

Request	Tenant	Queue Depth	Latency
r007	tenant-a	24	2410ms
r008	tenant-b	28	2650ms
r010	tenant-c	35	2810ms
r012	tenant-a	41	2960ms
r013	tenant-b	43	2720ms
r015	tenant-c	42	2540ms
r016	tenant-a	39	2290ms

4. Part D: System Design - Capacity Arithmetic

The numbers you MUST know:

Metric	Calculation	Result
Sustained rate	10,000 req/min / 60	~167 req/s
Burst rate	18,000 req/min / 60	300 req/s
Sustained concurrency	167 x 2s response time	334 concurrent
Burst concurrency	300 x 2s response time	600 concurrent
With 30% headroom	600 x 1.3	~780 slots
Gateway instances	780 / 100 per instance	8 instances
Burst excess requests	(18K-10K) x 2 min	16,000 extra

Why 2s response time for concurrency calc?

p95 SLO target is 2s. Capacity plan for the SLO target, not the median.

Why 100 concurrent per gateway instance?

Conservative for async I/O. Gateway is stateless HTTP routing. Bottleneck is upstream provider concurrency, not the gateway itself.

Tenant fairness:

High-traffic tenant: max 50% of capacity (390 slots during burst). Others: 25% each (195 slots). Soft limits - unused capacity spills. Only enforced under contention.

Fallback budget: 20% cap

Without cap, primary outage routes 100% to fallback -> fallback degrades -> cascading failure. 20% is a judgment call. In production, tune based on fallback's tested capacity.

First bottleneck expected:

Upstream provider concurrency limits. Trace data confirms: queue_depth climbing preceded errors.

5. Part F: Business Case - Unit Economics

The math:

Scale	Calculation	Monthly cost
Pilot (1,200 PRs/mo)	$1,200 \times 2.5 \times \0.18	\$540/month
6-week pilot total	$\$540 \times 1.5$	\$810 inference
Scale (30K PRs/mo)	$30,000 \times 2.5 \times \0.18	\$13,500/month
Annual at scale	$\$13,500 \times 12$	\$162,000/year

What's MISSING (they WILL ask this):

- Retry/failure cost: actual runs/PR may exceed 2.5
- False positive cost: reviewers waste time on spurious warnings. At \$1/min, 10 min per FP, 40% FP rate: ~\$48K/mo at scale = 3.5x inference cost
- Infrastructure cost: gateway, logging, storage not in \$0.18/run
- Self-hosted variable cost: \$30K setup stated, ongoing compute unknown
- Token volume variance: large PRs consume more tokens

Why hosted for pilot:

Self-hosted \$30K setup exceeds \$25K pilot budget. Validate value proposition first, optimize cost later. Self-hosted is Phase 2.

Strongest objection:

Pilot measures review SPEED but stakeholders care about ESCAPED DEFECTS - and no baseline connects the two. You can prove faster reviews, but proving BETTER reviews requires 3-6 months of post-merge defect data. Pilot can detect if it makes things WORSE, but the positive case is unproven in 6 weeks.

6. The Traps in the Assignment

[!] TRAP 1: Credential leak - provider_content_filter.json contains "authorization": "Bearer do-not-copy-this-field". If this appears in ANY output = security failure. Our code uses only hardcoded static messages.

[!] TRAP 2: Malformed data - provider_malformed_success.json has choices:[] (empty) and prompt_tokens:"unknown" (string). Must be invalidProviderResponse (502), not crash or silently coerce to zero.

TRAP 3: Nearest-rank vs linear interpolation. Spec says nearest-rank. Python's statistics.quantiles() uses linear interp = WRONG. We implement ceil(P/100 x N) manually.

TRAP 4: Fallback for rate limits. Tempting but spec says NO. Rate limiting = quota issue, not outage.

7. Likely Live Review Questions

Q: "Walk me through a 503 hitting the gateway."

A: `normalizeFailure` called with `upstreamStatusCode:503`. Checks payload codes first (none), then timeout (no), then matches 503 in 5xx range. Returns `CanonicalFailure: providerUnavailable, HTTP 503, retryable=true, failoverEligible=true`, static safe message. Then `decideFallback`: if different fallback configured and not tried yet, fallback permitted. Fallback success goes through `normalizeSuccess`. Product app never sees fallback happened.

Q: "What if I allow fallback on rate limits?"

A: Change `rateLimited`'s `failoverEligible` to true. Add `rateLimited` to permitted codes in `decideFallback`. But also add rate-limit-specific cooldown - if primary is rate-limiting us, fallback might too. Need per-provider backoff counter.

Q: "Why structural detection instead of provider type parameter?"

A: Method signature doesn't include provider type - that's a constraint of the given API. If designing the API, I'd add a `providerType` enum and dispatch to format-specific parsers. Structural detection is best available given the signature. Flagged for review in `self_assessment.md`.

Q: "p95 is 2810ms, above 2000ms SLO. Your read?"

A: This is an incident window, not steady-state. High p95 driven by fallback requests (2290-2960ms) during `queue_depth` spike. Normal operation (r001-r005, r019-r020) has 420-820ms. SLO breach is direct consequence of incident. Fix: prevent queue buildup, not accept higher SLOs.

Q: "How would you handle a new provider format?"

A: Add new branch checking for new provider's distinguishing key. But for 3+ formats, refactor to strategy pattern: map from provider type to format-specific parser class. More maintainable than structural detection.

Q: "Why does tenant-a have all the errors?"

A: FACT: tenant-a has 60% of requests (12/20). HYPOTHESIS: disproportionate load means more likely to hit saturated provider. CONFOUND: tenant-a's volume may have CAUSED saturation. Can't distinguish correlation from causation without knowing provider capacity limit and per-tenant rate over longer window.

Q: "Your fallback budget is 20%. How did you pick that?"

A: Judgment call, not computed. During total outage, 20% means 4/5 requests fail - bad, but 100% fallback risks cascading BOTH providers down, which is worse. In production, tune based on fallback provider's tested capacity. Make it a configurable parameter with alerting at 10%.

Q: "What if pilot budget were \$100K?"

A: Run three-arm trial: hosted ML vs self-hosted ML vs rule-based baseline simultaneously. Self-hosted \$30K setup fits. Compare all three in one pilot instead of sequencing. Also invest in proper UX (dedicated panel vs comment). Stronger recommendation from richer data.

8. Quick Reference Card

Key numbers at a glance:

Category	Value
Total requests	20
Success/error rate	80% / 20%
Observed RPS	9.3023
p50 / p95 / p99 / max latency	1210 / 2810 / 2960 / 2960 ms
Fallback count/rate	7 / 35%
Prompt / completion / total tokens	1987 / 762 / 2749
Sustained req/s	167 req/s
Burst req/s	300 req/s
Burst concurrency (with headroom)	780 slots
Pilot monthly cost	\$540/month
Scale monthly cost	\$13,500/month
Annual cost at scale	\$162,000/year

Files and what they do:

File	Lines	Key thing to know
gateway_normalizer.dart	~280	normalizeSuccess, normalizeFailure, decideFallback, cosineSimilarity
gateway_normalizer_test.dart	~300	43 tests: 2 original + 41 added
analyze_trace.py	~120	nearest_rank_percentile, summarize